

**NAME: P.KAVYA SAI**

**ROLL NUMBER: 2420030322**

**Linkedin post:** [Linkedin post](#)

### **Task 03 Documentation: Prompting for Task Automation**

**Program:** SkillCraft Technology

**Objective:** Apply prompt engineering principles to semi-automate a crucial business process or repetitive data-handling task. Design a clear, consistent system prompt capable of processing messy, unstructured data inputs and producing predictable, structured analytical blocks across multiple execution runs.

 **Core Lesson:** Processing complex customer data streams or business assets manually is highly inefficient. Building robust instructions with clear constraints allows an LLM to serve as a reliable automated pipeline for data transformation, data cleaning, and analytical parsing.

#### **System Prompt Design**

The core architecture of the automated data pipeline relies on a structured system prompt that isolates task parameters, defines data schemas, and outlines strict output boundaries:

Plaintext

You are a product feedback analysis tool. Your task is to analyze a raw customer review and generate a concise summary block.

Format your output exactly like this template:

**\*\*Sentiment:\*\*** [Positive / Negative / Neutral]

**\*\*Overall Rating:\*\*** [X/5 Stars - estimate based on text tone]

**\*\*Key Issues:\*\*** [Bulleted list of complaints, or "None" if positive]

**\*\*Praises:\*\*** [Bulleted list of compliments, or "None" if negative]

Keep all bullet points under 5 words each. Do not add any other commentary.

#### **Input-Output Execution Examples**

##### **Example 1: Customer Review Analysis Pipeline**

- **Unstructured Input File:** Customer Review Dataset.txt

**Sentiment:** Negative

**Overall Rating:** 2/5 Stars

**Key Issues:**

- Poor battery life
- Broken charging cable
- Misleading battery claims
- Hardware quality problems

**Praises:**

- Excellent sound quality
- Instant phone pairing
- Compact size performance

**Example 2: Financial Document Extraction (Text to JSON)**

- **Unstructured Input File:** Receipt.txt
- **Prompt Task:** Extract key information (merchant\_name, date, total\_amount, and items) from messy receipt text and output a clean, valid JSON object without conversational intros or markdown wrapper blocks.
- **Automated Structured Output:**

JSON

```
{"merchant_name":"STARBUCKS COFFEE #4422","date":"2026-04-12","total_amount":10.5,"items":["Grande Latte","Choco Croissant"]}
```

**Example 3: Meeting Notes Transformation (Conversational to Markdown Table)**

- **Unstructured Input File:** Works To Be Done.txt
- **Prompt Task:** Parse messy, conversational meeting transcripts and map the discussion into a structured four-column Markdown table outlining the Task, Assignee, Priority level, and Deadline metrics.

| Task                                     | Assignee | Priority | Deadline  |
|------------------------------------------|----------|----------|-----------|
| Fix the login bug preventing user access | Dave     | High     | Friday    |
| Update the investor meeting slide deck   | Sarah    | Low      | Next week |

| Task                          | Assignee | Priority | Deadline           |
|-------------------------------|----------|----------|--------------------|
| Order pizza for the hackathon | Mark     | Medium   | Thursday afternoon |

### Prompt Iteration, Optimization & Debugging

During the development and testing phases of this task automation system, several key behavioral anomalies were encountered and addressed through prompt debugging:

- **Handling Missing Fields:** In early iterations of the receipt extraction task , the model would throw formatting errors or leave fields out entirely if a specific item (like a missing date) wasn't present in the source text. Adding the explicit fallback instruction—*"If a field cannot be found, set its value to null"*—stabilized the system pipeline.
- **Suppressing Conversational Fillers:** When generating data intended for programmatic consumption (like programmatic JSON strings or raw Markdown tables), downstream parsers fail if the AI adds introductory pleasantries (e.g., *"Here is the JSON object you requested:"*). To combat this, strict structural formatting barriers were applied: *"Do not include markdown code blocks, intro text, or explanation."*
- **Enforcing Strict Length Constraints:** In the customer feedback model ,bullet points initially expanded into verbose sentences. Adding the exact boundary constraint—*"Keep all bullet points under 5 words each"*—forced the LLM to summarize cleanly, ensuring clear insights at a single glance.